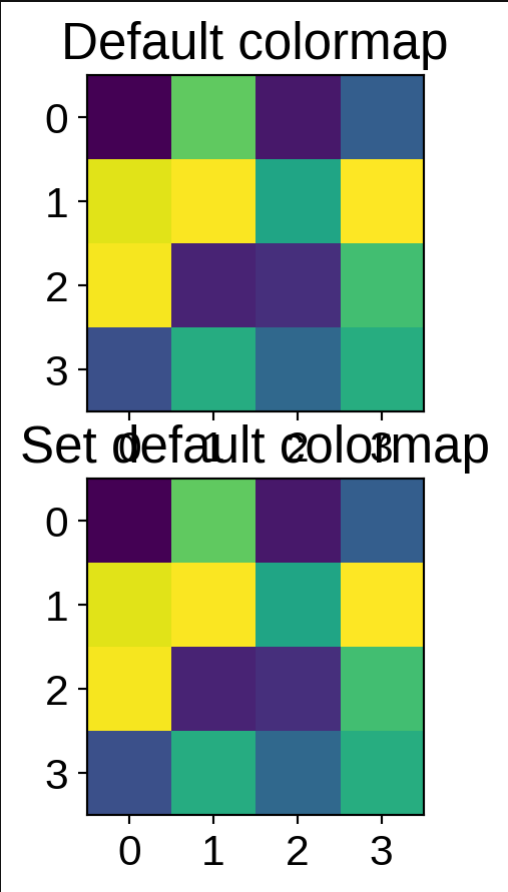


# CPSC 330 Lect

## DBSCAN

## Hierarchical

## Clustering



# iClicker Exercise 15.1

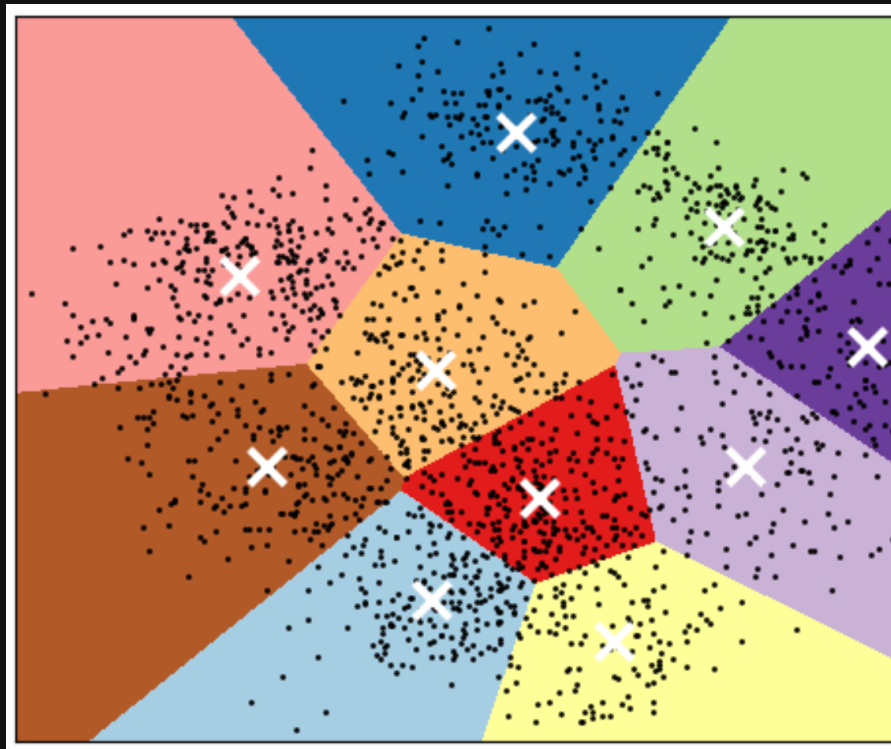
Select all of the following statements that are TRUE.

- A. With  $n$  examples,  $K$  clusters, and  $d$  dimensions, K-Means learns  $K$  cluster centers, each with  $d$  coordinates.
- B. The meaning of  $k$  in k-nearest neighbors is very different from the meaning of  $k$  in K-Means clustering.
- C. Scaling of input features is crucial in K-Means clustering.
- D. In clustering, it's almost always a good idea to have a large number of equal-sized clusters.

# Limitations of means

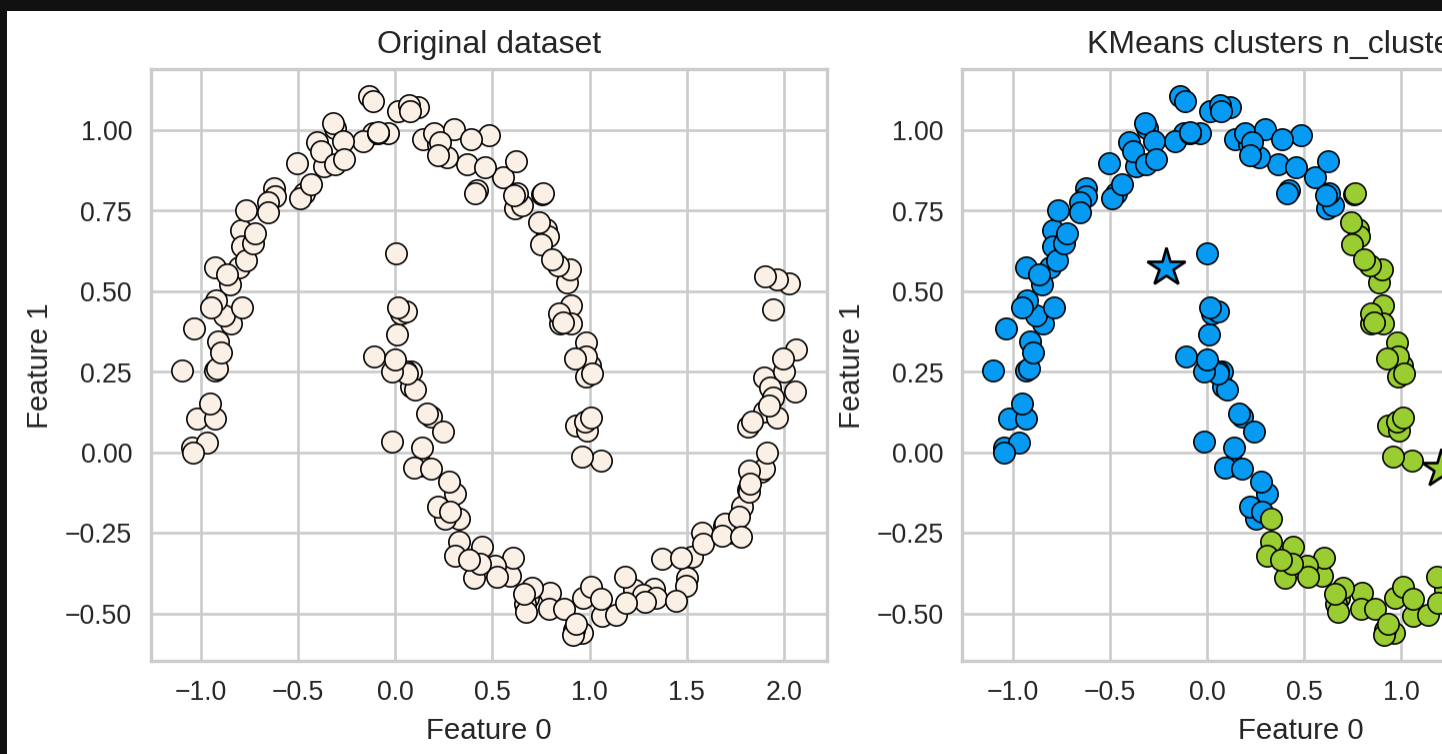
# Shape of clusters

- Good for spherical clusters of more c



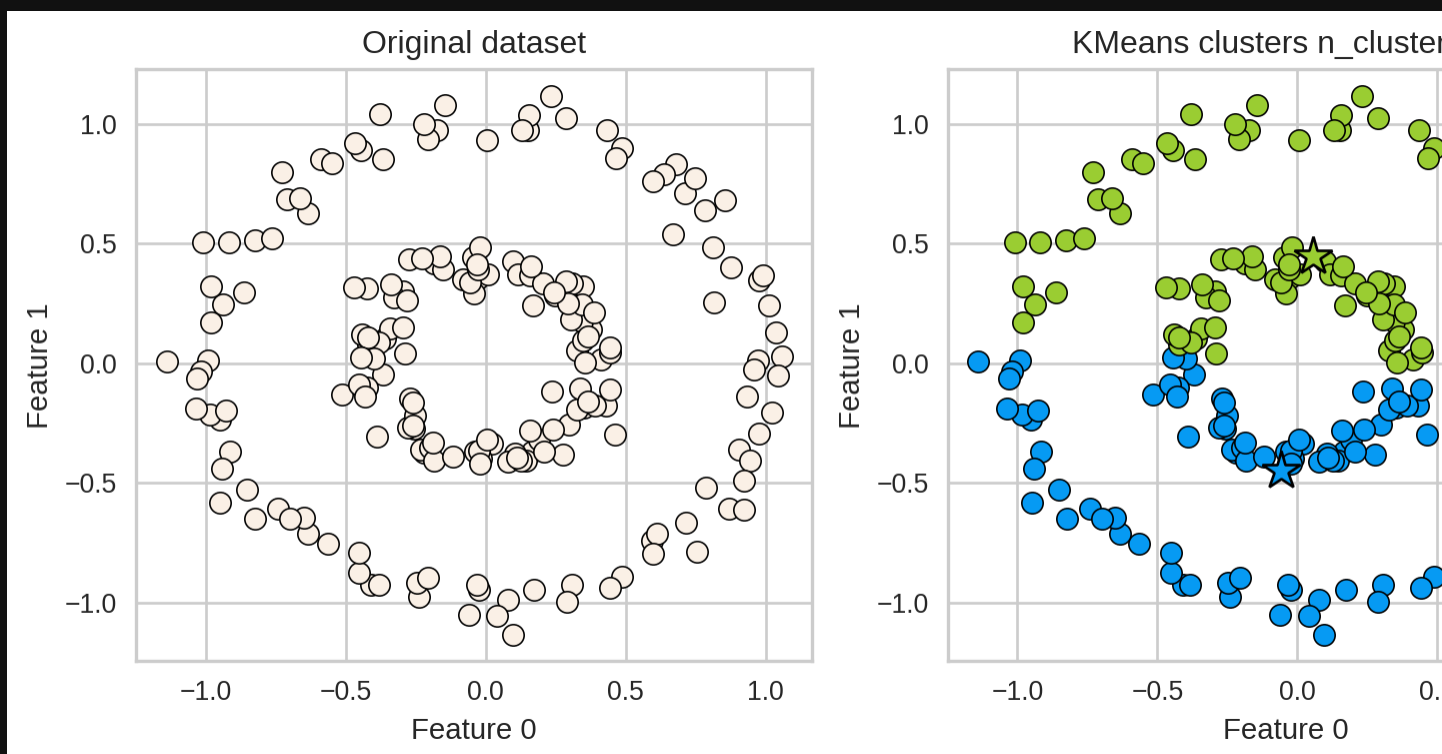
# K-Means: failure case 1

- K-Means performs poorly if the clusters have complex shapes (e.g., two moons data)



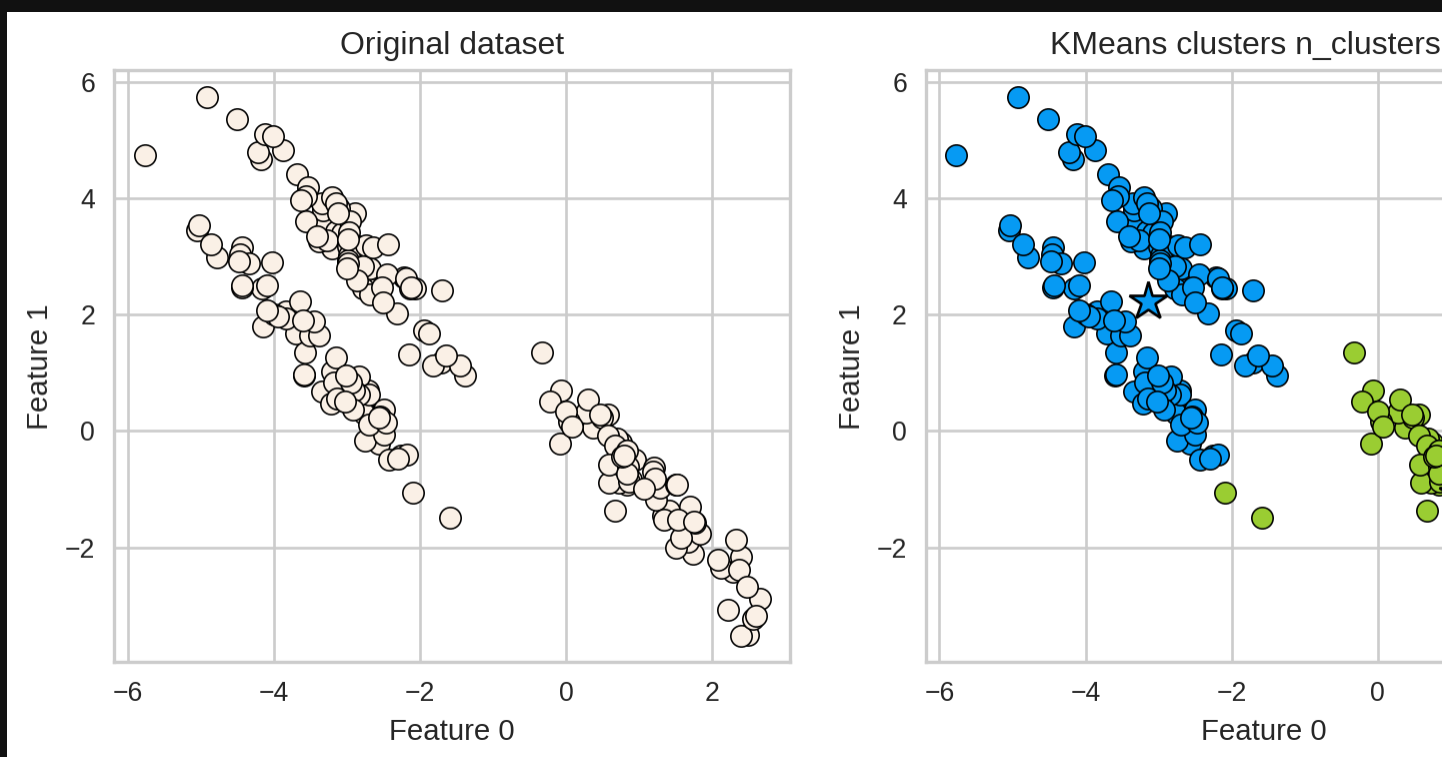
# K-Means: failure case 2

- Again, K-Means is unable to capture shapes.



# K-Means: failure case 3

- It assumes that all directions are equal for each cluster and fails to identify non-spherical clusters.



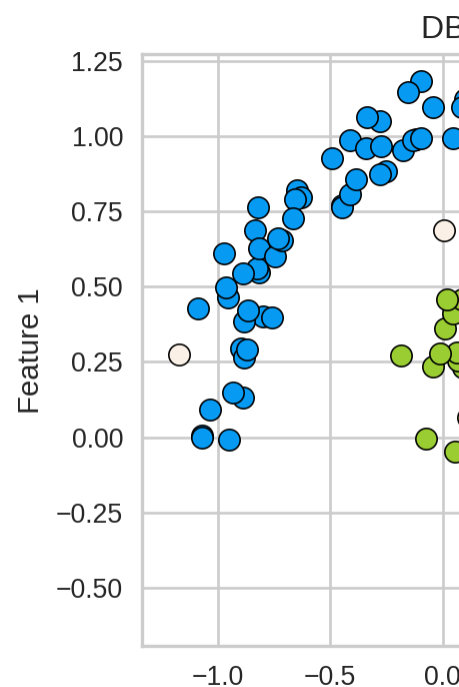
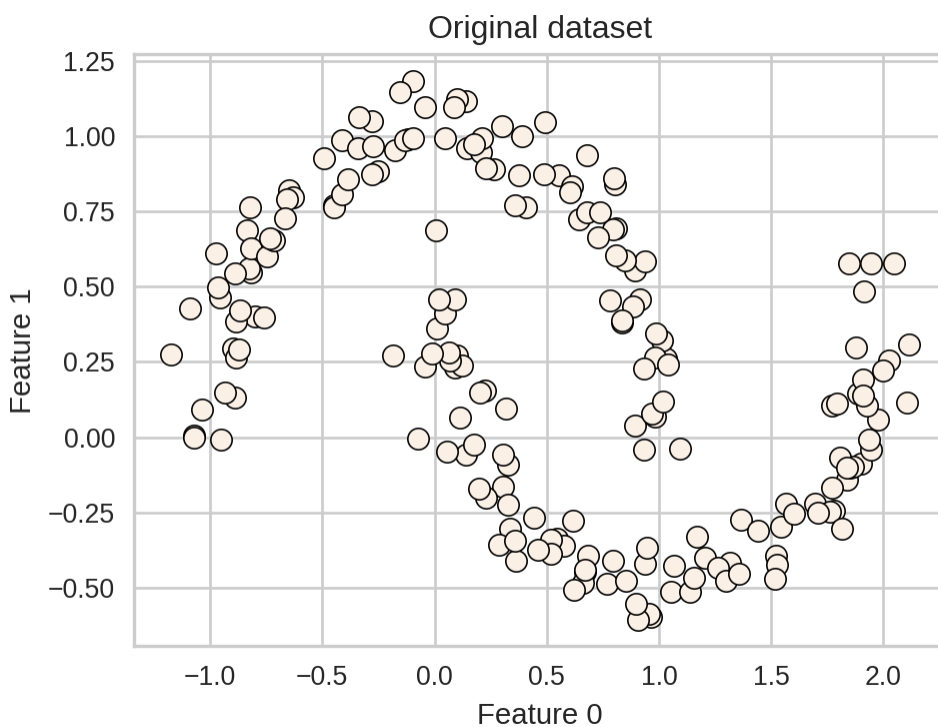


**Can we do better**

# DBSCAN

- **D**ensity-**B**ased **S**patial **C**lustering of **A**pplications with **N**oise
- A density-based clustering algorithm

```
1 X, y = make_moons(n_samples=200, noise=0.08, random_state=42)
2 dbscan = DBSCAN(eps=0.2)
3 dbscan.fit(X)
4 plot_original_clustered(X, dbscan, dbscan.labels_)
```

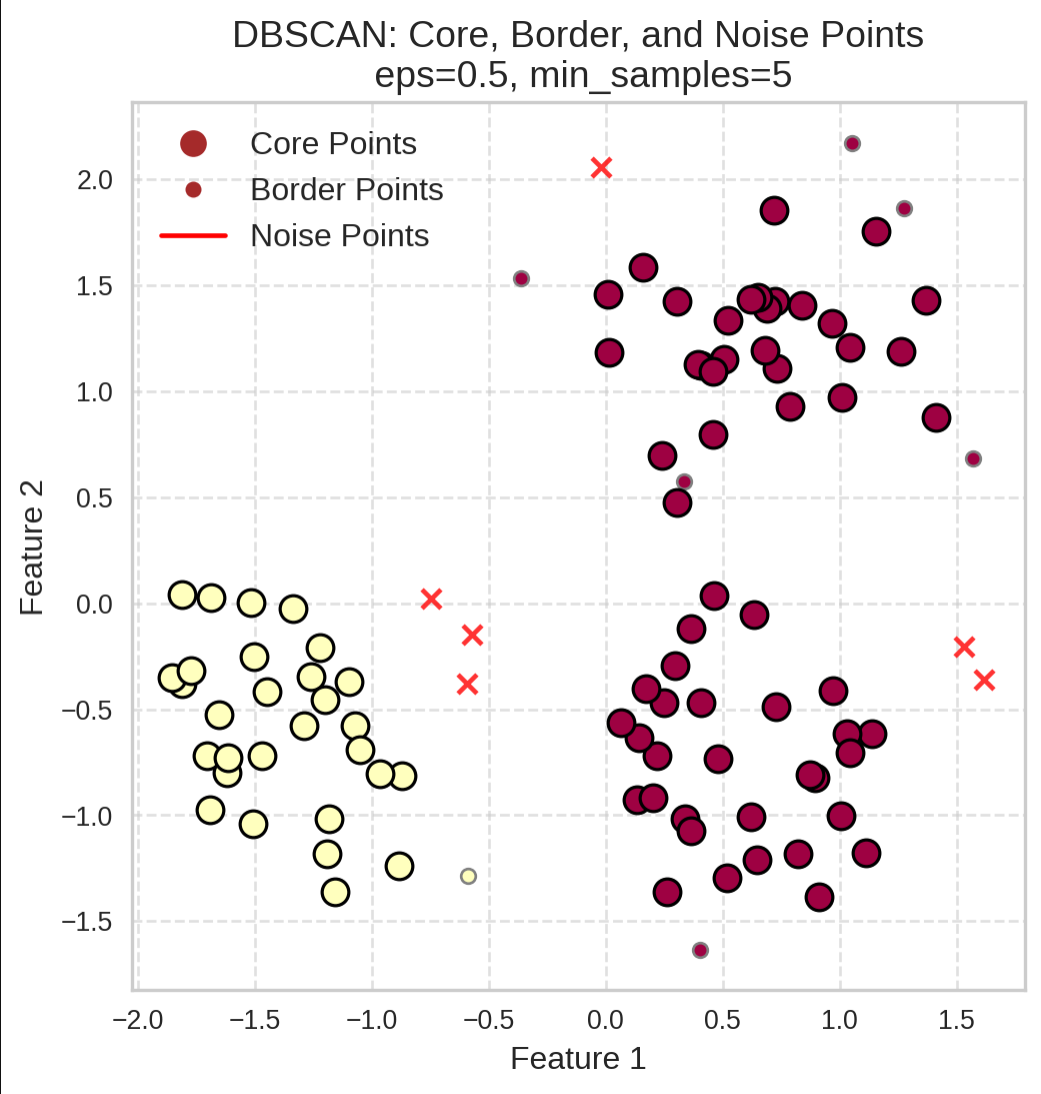


# Two main hyperparam

In order to identify dense regions, we need two hyperparameters:

- **eps**: determines what it means for points to be close
- **min\_samples**: determines the number of **neighbouring points** we require to form a cluster  
for a point to be part of a cluster

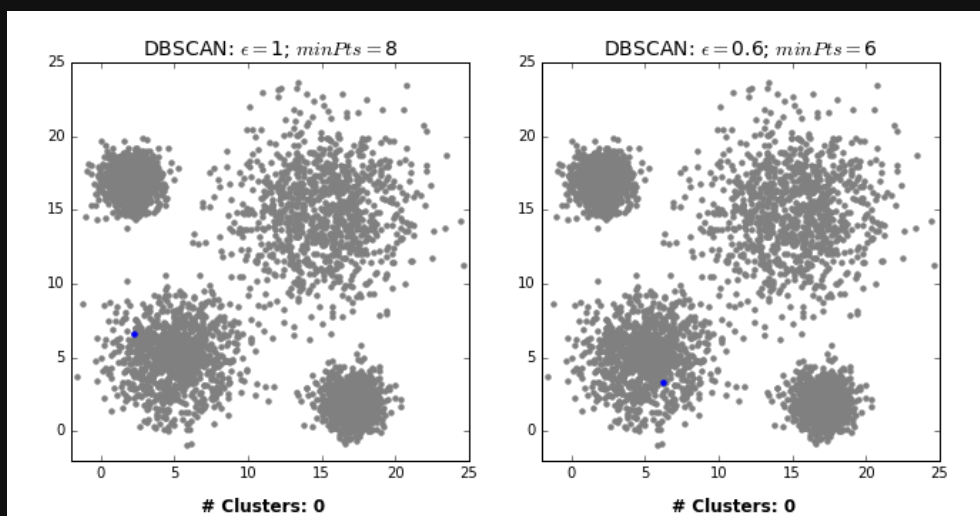
# DBSCAN analogy



Consider  
context

- Social  
point
- Friend  
who  
Border
- Lone

# DBSCAN algorithm



- Pick a point  $p$
- Check whether  $p$  is a core point or not.
- If  $p$  is a core point, label it as a core point (label).
- Spread the label to its  $\epsilon$ -neighbours.
- Check if any of its neighbours received the label. If yes, spread the label to its neighbours as well.
- Once there are no more core points left to spread the label, assign all unlabeled points to noise.

# Clustering Algorithm V

Here is a really nice web-app to visualize  
[clustering-visualizer.web.app/dbscan](https://clustering-visualizer.web.app/dbscan)

# Human DBSCAN act

## Goal

Experience how  $\epsilon$  (**eps**) and **min\_samp** counts as a cluster.

# Setup

- Each student = one **data point**
- Distance (eps) = **arm's length (~1 m)**
- Each gets a **sticky note** to mark cluster **marker** (colour)
- Start from a random student: check v



# Two parallel runs

| Side of Class | $\epsilon$ (eps)       | min_samples |
|---------------|------------------------|-------------|
| Left side     | ~1 m<br>(arm's length) | 5           |
| Right side    | ~1 m<br>(arm's length) | 15          |

Side of  
Class

$\epsilon$  (eps)

min\_samples

# How to play

1. Pick a **starting student**.
2. Count how many of your neighbours distance.
3. If you have  $\geq$  **min\_samples**, you are start spreading your colour.
4. Neighbours who receive a colour but **border points**.
5. Students who never receive a colour **points**.

# After the activity

- Which side formed **more clusters**?
- What happened when **min\_samples**
- Why doesn't DBSCAN need to know the number of clusters  $k$ ?

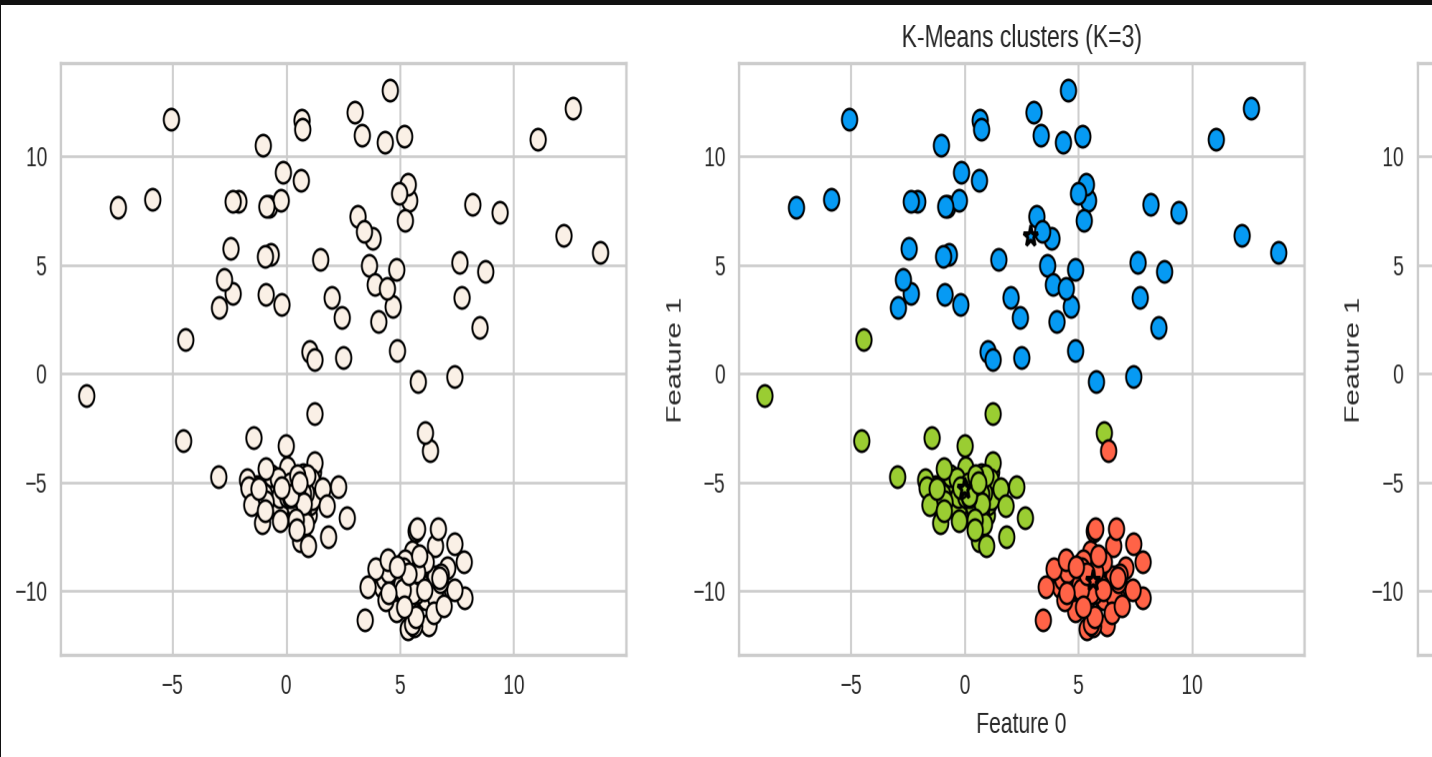
# How to tune eps and mi

- Can you use GridSearchCV and Random

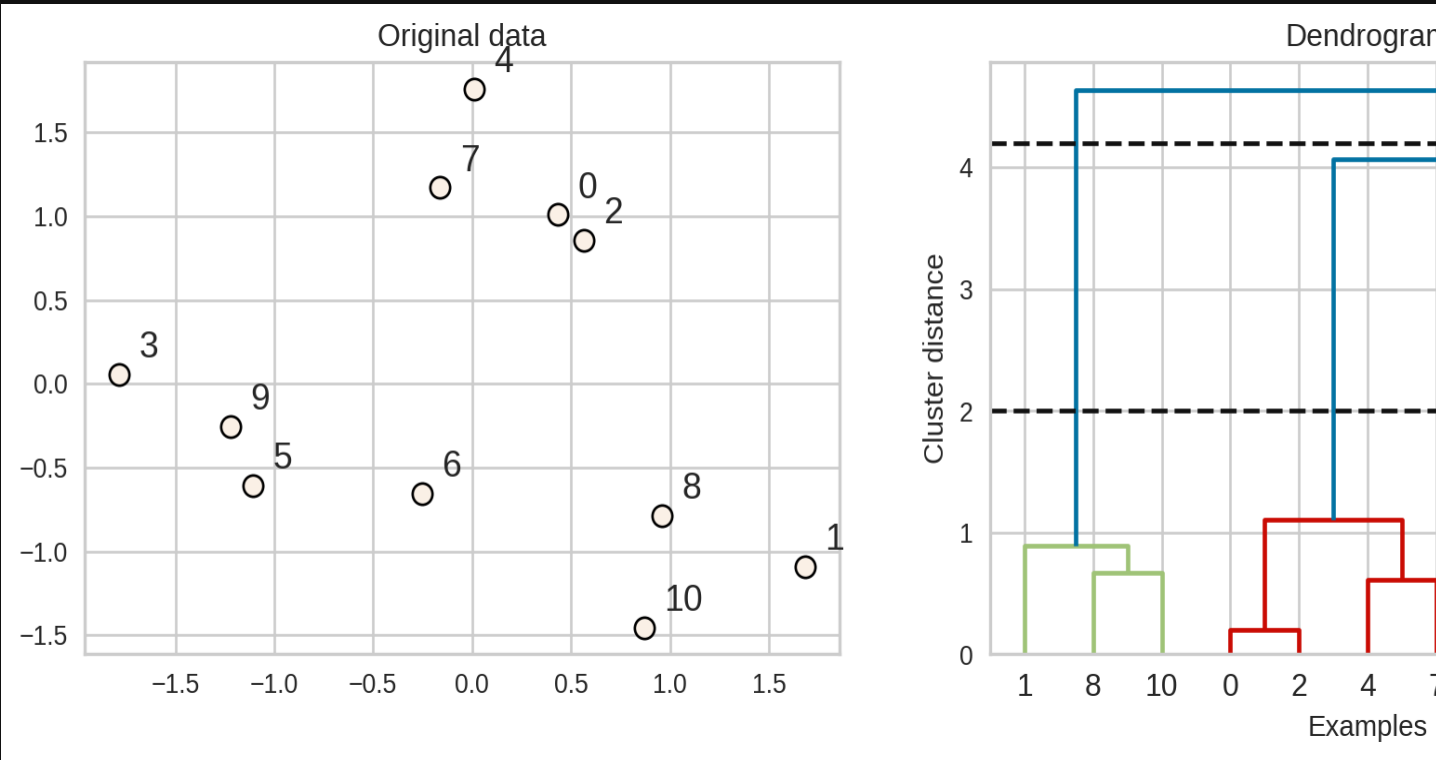
# DBSCAN: failure cases

- Let's consider this dataset with three clusters of varying density
- K-Means performs better compared to DBSCAN. But knowing the value of  $K$  in advance.

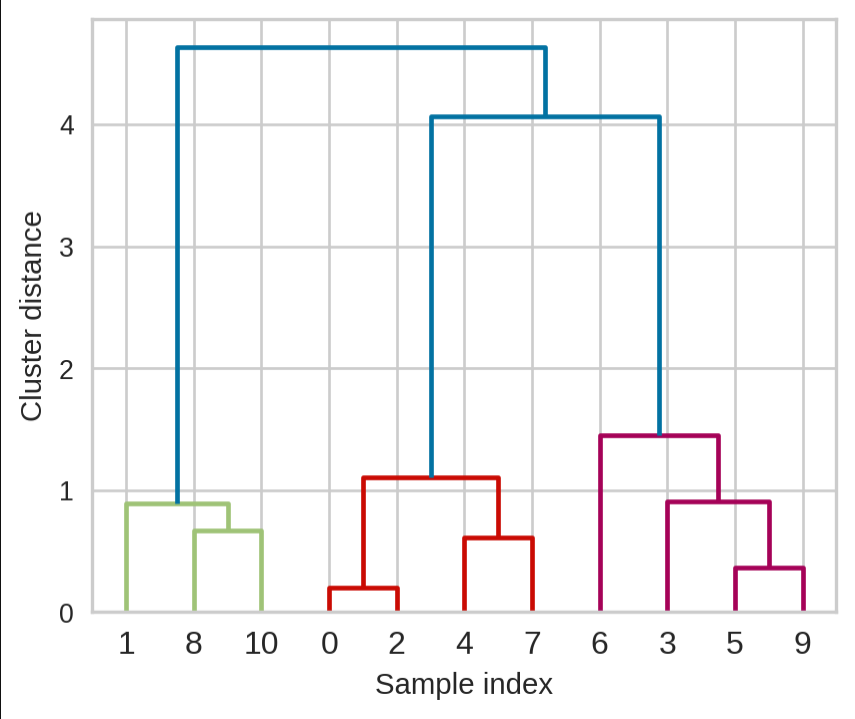
[ 0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15]



# Hierarchical clustering



# Dendrogram



- De tre
- On ha
- On ha be

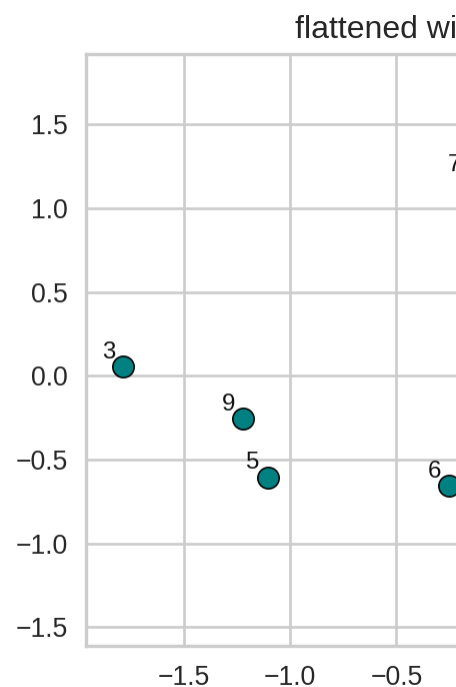
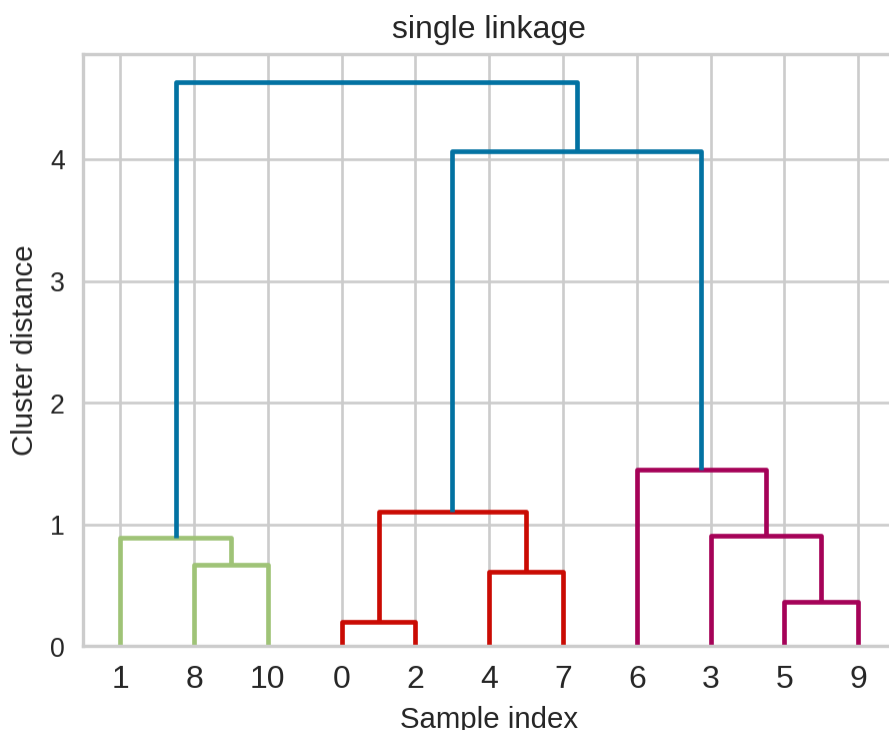


# Flat clusters

- This is good but how can we get cluster dendrogram?
- We can bring the clustering to a “flat”  
`fcluster`

# Flat clusters

```
1 from scipy.cluster.hierarchy import fcluster
2 # flattening the dendrogram based on maximum number of clusters
3 hier_labels1 = fcluster(linkage_array, 3, criterion='maxclust')
4 plot_dendrogram_clusters(X, linkage_array, hier_labels1)
```



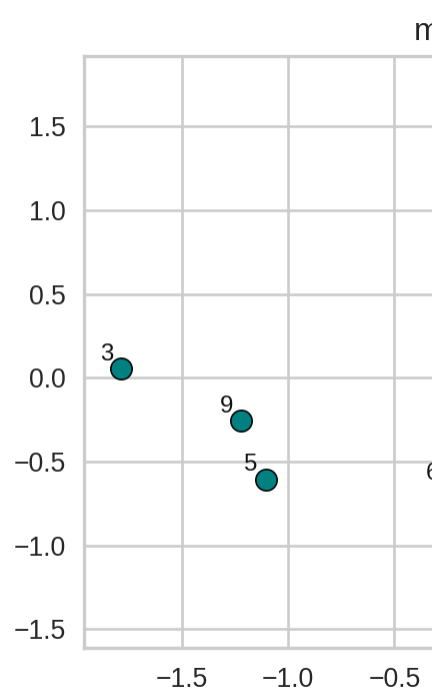
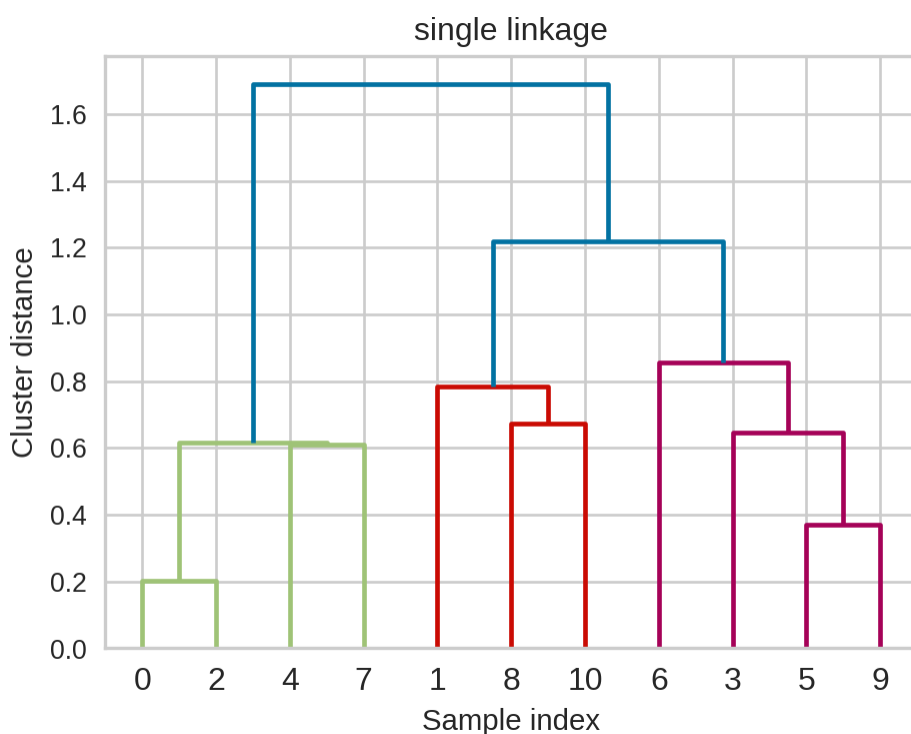
# Linkage criteria

- When we create a dendrogram, we need to calculate distances between clusters. How do we measure distances between clusters?
- The **linkage criteria** determines how to find similarity between clusters
- Some example linkage criteria are:
  - Single linkage → smallest minimal distance, leads to chain clusters
  - Complete linkage → smallest maximum distance, leads to compact clusters
  - Average linkage → smallest average distance between clusters
  - Ward linkage → smallest increase in within-cluster variance, leads to balanced clusters

# Example: Single linkage

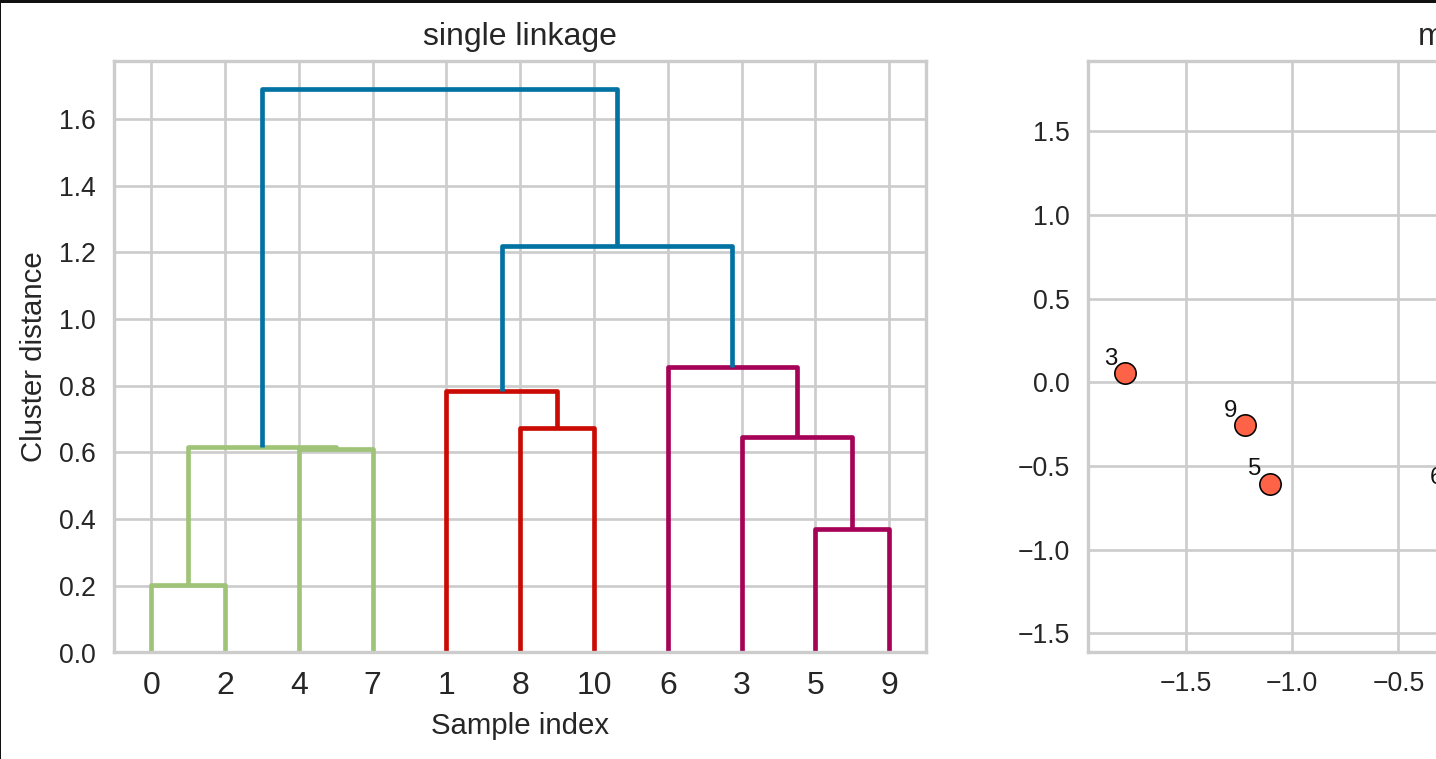
Suppose you want to go from 3 clusters to 2 clusters. What should you merge?

```
1 X_orig, y = make_blobs(random_state=0, n_samples=11)
2 X = StandardScaler().fit_transform(X_orig)
3 linkage_array = single(X)
4 hier_labels = fcluster(linkage_array, 3, criterion="maxclust")
5 plot_dendrogram_clusters(X, linkage_array, hier_labels, title="maxclust 3")
```



# Example: Single linkage

```
1 hier_labels = fcluster(linkage_array, 2, criterion="maxclust")
2 plot_dendrogram_clusters(X, linkage_array, hier_labels, title="maxclust 2")
```



# iClicker Exercise 15.2

Select all of the following statements which are True

- A. In hierarchical clustering we do not have to worry
- B. Hierarchical clustering can only be applied to small  
dendrograms are hard to visualize for large datasets
- C. In all the clustering methods we have seen (K-Means  
clustering), there is a way to decide the number of
- D. To get robust clustering we can naively ensemble (the  
the most popular label) produced by different clusters
- E. If you have a high Silhouette score and very clean  
means that the algorithm has captured the semantics  
of our interest.

# Activity

Discuss the following

| Clustering Method | KMeans | DBSCAN |
|-------------------|--------|--------|
| Approach          |        |        |
| Hyperparameters   |        |        |
| Shape of clusters |        |        |
| Handling noise    |        |        |
| Distance metric   |        |        |

# Discussion question

Which clustering method would you use for the scenarios below? Why? How would you preprocess the data in each case?

- Scenario 1: Customer segmentation
- Scenario 2: An environmental study to identify clusters of a rare plant species
- Scenario 3: Clustering furniture items for inventory management and customer recommendations



# Break

Let's take a break!



# Group Work: Class Demo

## Coding

**Super cool Demo!**

For this demo, each student should **click** create a new repo in their accounts, then clone the repo locally to follow along with the demo.

All credit to Dr. Varada Kolhatkar for putting this together!